

Experiment No. 1

Roll No.	
Date	/ /

(1) write a program to declare class ~~number~~ students having data members as roll no, name, age & display data for 1 object.

```
#include <iostream>
using namespace std;
class student
{
    int roll;
    String name;
    public:
    void accept()
    {
        cout << "Enter value of roll, name";
        < in >> roll >> name;
    }
    void display()
    {
        cout << "roll : " << roll << " name : " << name
        << endl;
    }
};

int main()
{
    Student s1;
    s1.accept();
    s1.display();
    return 0;
}
```

(2) write a program to declare a class book having data members as Book name, price, no of pages, accept for 2 objects & display the name of Book having greater price

```
#include <iostream>
using namespace std;
class book
```

```
{
    int pages;
    String name;
    float price;
    public:
    void accept()
    {
```

```
        cout << "Enter value of name, price, pages" << endl;
        cin >> name >> price >> pages;
    }
```

```
    void display()
    {
```

```
        cout << "Name:" << name << ", Price:" << price << endl;
        cout << "pages:" << pages << endl;
    }
```

```
};

int main()
```

```
{
    book b1, b2;
    b1.accept();
    b2.accept();
```

```
    cout << "Book with greater price:"
```

(2) write a program to declare a class book having data members as Book name, price, no of pages, accept for 2 objects & display the name of Book having greater price

```
#include <iostream>
using namespace std;
class book
```

```
{
    int pages;
    String name;
    float price;
    public:
    void accept()
    {
```

```
        cout << "Enter value of name, price, pages" << endl;
        cin >> name >> price >> pages;
    }
```

```
    void display()
    {
```

```
        cout << "Name:" << name << ", Price:" << price << endl;
        cout << "pages:" << pages << endl;
    }
```

```
};

int main()
```

```
{
    book b1, b2;
    b1.accept();
    b2.accept();
```

```
    cout << "Book with greater price:"
```

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----

- ```
#include <iostream>
```

class city

2

Being aware.

\_\_\_\_\_

void accept()

5

- in 32 same 32 population

2

• 74

11

2

10

city (LS)

$$\frac{\min_i \{x_i\}}{\max_i \{x_i\}}$$

```
for (i = 0; i < 5; i++)
```

2

```
 C.E.D : accept C;
```

2

more = c [O] population

for (i=0; i<5; i++)

(2) Write a program to declare a class 'Account' having data member as account no & balance. Accept this data for 10 accounts & give interest of 10% where balance is equal or greater than 5000 & display them.

```
#include <iostream>
using namespace std;
class Account
```

```
{
public:
 int acc no;
 float bal;
 String name;
```

```
void accept();
```

```
{
cout << "Enter name of customer:";
cin >> name;
cout << "Enter account number:";
cin >> acc no;
cout << "Enter balance:";
cin >> bal;
```

```
}
};
int main()
```

```
{
 Account a[10];
 for (int i = 0; i < 10; i++)
```



C8) write a program to declare a class 'staff' having data members as name & post. Accept this data for 5 staff & display names of staff who are "Hob".

```
#include <iostream>
#include <string>
using namespace std;
class staff
{
 char name;
 char post;
public:
 void accept()
 {
 cout << "Enter name:";
 cin >> name;
 cout << "Enter post:";
 cin >> post;
 }
 void display()
 {
 if (strcmp(post, "Hob") == 0)
 {
 cout << "Hob name: " << name << endl;
 }
 }
};
int main()
{
 staff s;
```

C8) write a program to declare a class 'staff' having data members as name & post. Accept this data for 5 staff & display names of staff who are "Hob".

```
#include <iostream>
#include <string>
using namespace std;
class staff
{
 char name;
 char post;
public:
 void accept()
 {
 cout << "Enter name:";
 cin >> name;
 cout << "Enter post:";
 cin >> post;
 }
 void display()
 {
 if (strcmp(post, "Hob") == 0)
 {
 cout << "Hob name: " << name << endl;
 }
 }
};
int main()
{
 staff s;
```

```

 cout << "List details for Staff" <<
 i + 1 << "i" << endl;
 STD::accept c3;
}
cout << "In List of Staff with post " <<
 << endl;
for(int i=0; i<5; i++)
{
 STD::display if HOD();
}
return 0;
}

```

output:

details for Staff 1:

Name: James

Post: HOD

Details for Staff 2:

Name: Charles

Post: lecturer

Details for Staff 3:

Name: Henry

Post: principle

Details for Staff 4:

Name: William

Post:



Details for Staff 5

Name: Oliver

Post: Director

List of MOD'S

Name: James

Name: William

26/8

### Experiment No. 3

c1) write a program to declare a class containing data members as title, author, name, price Accept & display the information for one object using pointer to a

```
#include <iostream>
using namespace std;
class book
```

```
{
```

```
private:
```

```
String book - title;
```

```
String author name;
```

```
float price;
```

```
public:
```

```
void accept();
```

```
{
```

```
cout << "Enter book - title";
```

```
cin >> book - title;
```

```
cout << "Enter author name";
```

```
cin >> author name;
```

```
cout << "Enter price";
```

```
cin >> price;
```

```
}
```

```
void display()
```

```
{
```

```
cout << "book title = " << book - title;
```

```
cout << "author name = " << author
```

```
cout << "price = " << price;
```

```
}
```

(3) write a program to demonstrate the use of nested class.

```
#include <iostream>
```

```
using namespace std;
```

```
class Student {
```

```
 int roll;
```

```
 String name;
```

```
public:
```

```
 void accept() {
```

```
 cout << "Enter roll & name";
```

```
 cin >> roll >> name;
```

```
 }
```

```
 void display() {
```

```
 cout << "Name & roll no is: " << name
```

```
 << endl;
```

```
 }
```

```
class Marks {
```

```
 int m1, m2, avg;
```

```
public:
```

```
 void accept() {
```

```
 cout << "Enter marks m1 & m2";
```

```
 cin >> m1 >> m2;
```

```
 }
```

```
 void calculate_avg() {
```

```
 avg = (m1 + m2) / 2;
```

```
 }
```

```
 void display() {
```

```
 cout << "Average of marks is: " << avg
```

(3) write a program to demonstrate the use of nested class.

```
#include <iostream>
```

```
using namespace std;
```

```
class Student {
```

```
 int roll;
```

```
 String name;
```

```
public:
```

```
 void accept() {
```

```
 cout << "Enter roll & name";
```

```
 cin >> roll >> name;
```

```
 }
```

```
 void display() {
```

```
 cout << "Name & roll no is: " << name
```

```
 << endl;
```

```
 }
```

```
class Marks {
```

```
 int m1, m2, avg;
```

```
public:
```

```
 void accept() {
```

```
 cout << "Enter marks m1 & m2";
```

```
 cin >> m1 >> m2;
```

```
 }
```

```
 void calculate_avg() {
```

```
 avg = (m1 + m2) / 2;
```

```
 }
```

```
 void display() {
```

```
 cout << "Average of marks is: " << avg
```

|      |    |
|------|----|
| DATE |    |
| PAGE | 11 |

```

3
9;
2;
int main()
{
 Student S1;
 S1.accept();
 S1.display();
 Student :: marks &n;
 &n.accept();
 &n.calculate_avg();
 &n.display();
 return 0;
}

```

Output

Enter roll & name: 67 Alpha

Alpha

Name & roll no is : 67 Alpha

Enter marks:

m1 = 89

m2 = 79

Average is : 84

26/8



# Experiment No. 4

C1) WAP to Swap two numbers of same class using object as function argument write function as member function.

```
#include <iostream>
using namespace std;
class Number {
 int num;
public:
 void accept() {
 cout << "Enter number:";
 cin >> num;
 }
 void display() {
 cout << "Number:" << num << endl;
 }
 void Swap (Number & obj) {
 int temp = num;
 num = obj.num;
 obj.num = temp;
 }
};

int main() {
 Number n1, n2;
 cout << "Enter first number:" << endl;
 n1.accept();
 cout << "Enter second number:" << endl;
 n2.accept();
 n1.Swap(n2);
}
```

```

cout << "In After swap";
cout << " First";
n1 display();
cout << " Second";
n2 display();
return 0;

```

Output

Enter first number:

enter number: 6

Enter second number:

enter number: 9

After swap:

first number: 9

second number: 6

(2) WAP to swap two numbers from class using concept of friend func

```

#include <iostream>

```

```

using namespace std;

```

```

class temp {

```

```

 int x, y, z;

```

```

public:

```

```

 void accept();

```

```

 {

```

```

 cout << "Enter two numbers";

```

```

 cin >> x >> y;

```

```

 }

```

```

 void display()

```

cout << "Enter two numbers";  
cin

cout << "After swap x is: " << x;  
cout << "After swap y is: " << y;

};  
friend void swap(t& t1, t& t2);

};  
void swap(t& t1, t& t2)

{  
t x = t1.x;  
t1.x = t2.x;  
t2.x = x;  
t y = t1.y;  
t1.y = t2.y;  
t2.y = y;  
}

};  
int main()

{  
t t1, t2;

t1.x = 1;  
t1.y = 2;  
t2.x = 3;  
t2.y = 4;

swap(t1, t2);

t1.display();

t2.display();

return 0;

}

Output :

Enter two numbers :

6

After swap x is: 6

After swap y is: 9

c3) WAP To swap two numbers from different class, using concept of friend function.

```
#include <iostream>
using namespace std;
class B;
class A {
 int numA;
public:
 void accept() {
 cout << "Enter number A: ";
 cin >> numA;
 }
 void display() {
 cout << "Number A = " << numA << endl;
 }
 friend void swapNumbers(A &, B &);
};
class B {
 int numB;
public:
 void accept() {
 cout << "Enter number B: ";
 cin >> numB;
 }
 void display() {
 cout << "Number B = " << numB << endl;
 }
 friend void swapNumbers(A &
```

```
void swap_numbers (A &a, B &b) {
 int temp = a.numA;
 a.numA = b.numB;
 b.numB = temp;
}
```

```
int main() {
 A a1;
 B b1;
```

```
 a1.accept();
 b1.accept();
```

```
 swap_numbers (a1, b1);
```

```
 cout << "In After Swapping: " << endl;
```

```
 a1.display();
```

```
 b1.display();
```

```
 return 0;
```

```
}
```

output

Enter number A: 67

Enter number B: 89

After Swapping:

Number A = 89

Number B = 67



(4) WAP to create two classes result 1 & result 2, which stores the marks of student. Read the value of a marks for both class objects & compute the avg of 2 results.

```
#include <iostream>
```

```
using namespace std;
```

```
class result1;
```

```
class result2;
```

```
int a;
```

```
public:
```

```
void accept() {
```

```
cout << "Enter marks out of 50
```

```
cin >> a;
```

```
}
```

```
friend void cal (result1, result2);
```

```
};
```

```
class result2 {
```

```
int b;
```

```
public:
```

```
void accept() {
```

```
cout << "Enter marks out of 50
```

```
cin >> b;
```

```
}
```

```
friend void cal (result1, result2
```

```
r2);
```

```
};
```

```
void cal (result1, result2, r2,
```

vs  
 $\text{float avg} = (\text{float}) \cdot (\text{int a} + \text{int b})$   
 cout << "In average: " << avg << endl;

int main()

{

result 1x;

result 2y;

x = accept();

y = accept();

Calc(x, y);

}

output: adding

Enter marks out of 50: 45

Enter marks out of 50: 46

Average = 45.5

  
 26/8

## Experiment 5

Write a C++ program to implement types of constructors.

- a) Write a program to find the sum of numbers between 1 to n using a constructor where the value of n will be passed to the constructor.

Ans

```
#include <iostream>
using namespace std;
```

```
class SumS
{
private:
public:
 int n, i, add = 0;
 SumS() {}
 SumS(int n) {
 n = n;
 add = n * (n + 1) / 2;
 }
 void display() {
 cout << "Sum = " << add << endl;
 }
 int main() {
 Sum S1;
 S1.display();
 return 0;
 }
}
```

```
class SumS
{
private:
public:
 int n, i, add = 0;
 SumS(int n) {
 n = n;
 add = n * (n + 1) / 2;
 }
 void display() {
 cout << "Sum = " << add << endl;
 }
 int main() {
 Sum S1(5);
 S1.display();
 return 0;
 }
}
```

```
class Sum
{
private:
public:
 int i, n, add;
 Sum(int n) {
 n = n;
 add = n * (n + 1) / 2;
 }
 void display() {
 cout << "Sum = " << add << endl;
 }
 int main() {
 Sum S1(5);
 S1.display();
 return 0;
 }
}
```

2. write a program to declare a class "student" having data members as name and percentage. write a constructor to initialize these data members. Accept ~~data~~ data and display data for one student.

Ans // default

```
#include <iostream>
using namespace std;
```

```
class student{
```

```
private:
```

```
public:
```

```
float per;
```

```
String name;
```

```
student() {
```

```
per = 94.5;
```

```
name = "Alpha";
```

```
}
```

```
void display() {
```

```
cout << "Name = " << name << "
```

```
percentage = " << per << " " << endl
```

```
}
```

```
}
```

```
int main() {
```

```
student s1;
```

```
s1.display();
```

```
return 0;
```

```
}
```

Q2

3) Define a class 'collego' member variables as roll no, name, WAF using constructor with default value as 20 computer Engineering, course accept this data for the object of class and display the data

// parameterized

```
#include <iostream>
using namespace std;
```

```
class Student2
```

```
private:
```

```
public:
```

```
float per;
```

```
String name;
```

```
Student (float p, String n) {
```

```
per = p;
```

```
name = n;
```

```
}
```

```
void display () {
```

```
cout << "Name = " << name << " & per = " << per << " . " << endl;
```

```
}
```

```
}
```

```
int main () {
```

```
Student s1 (93.8, "Alpha");
```

```
s1.display();
```

```
return 0;
```



// copy

```
class Student {
```

```
private:
```

```
public:
```

```
float per;
```

```
String name;
```

```
Student (float p, String n) {
```

```
per = p;
```

```
name = n;
```

```
}
```

```
Student (Student & s1) {
```

```
per = s1.per;
```

```
name = s1.name;
```

```
}
```

```
void display() {
```

```
cout << "name = " << name << " & percent = "
```

```
<< per << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
Student s1(93.5, "Alpha");
```

```
Student s2(s1);
```

```
s1.display();
```

```
s2.display();
```

```
return 0;
```

43. Def<sup>n</sup> class college, data member  
roll, name & course.

54 // default

```
#include <iostream>
using namespace std;
```

```
class college {
```

```
private:
```

```
public:
```

```
int roll_no;
```

```
String name;
```

```
String course;
```

```
college() {
```

```
course = "computer Engineering";
```

```
}
```

```
void accept() {
```

```
cout << "Enter name = ";
```

```
cin >> name;
```

```
cout << "Enter Roll no. = ";
```

```
cin >> roll_no;
```

```
}
```

```
void display() {
```

```
cout << "Name = " << name << "Roll no.
```

```
<< roll_no << " & course = " << course <<
```

```
<< endl; }
```

```
}
```

```
}
```

```
int main() {
```

```
college c1, c2;
```

```

cout << "Enter the student 1";
c1.accept(c);
cout << "stud ... 2";
c1.accept(c);
c2.display(c);
return 0;
}

```

b -> // parameterized

```

#include <iostream>
using namespace std;

```

```

class college {

```

```

private:

```

```

public:

```

```

int roll;

```

```

string name course;

```

```

college (int r, string n) {

```

```

roll = r;

```

```

name = n;

```

```

course = "computer eng";

```

```

}

```

```

void display () {

```

```

cout << "Name = " << name << "Roll = "

```

```

<< " & course = " << course << endl;

```

```

}

```

```

}

```

```

int main () {

```

```

college c (52, "Alpha");

```

```

c1 display c2;
c2 display c2;
return 0;
}

```

c3 11 copy

```

#include <iostream>
using namespace std;

```

```

class College {

```

```

private:

```

```

public:

```

```

int roll;

```

```

String name, course;

```

```

College (int r, String n) {

```

```

 roll = r;

```

```

 name = n;

```

```

 course = "computer engn ---";
}

```

```

College (College & c) {

```

```

 roll = c.roll;

```

```

 name = c.name;
}

```

```

void display () {

```

```

 cout << "name = " << name << "roll"

```

```

 << roll << "course = " << course;

```

```

 endl; } }

```

```

int main () {

```

```

 College c1 (82, "Alpha");
}

```

|          |     |
|----------|-----|
| Page No. |     |
| Date     | 1/1 |

```
cout << name << " " << name << "roll = "
<< roll << endl;
```

```
29
```

```
30;
```

```
int main () {
```

```
 student s1;
```

```
 student s2 (54, "Alpha");
```

```
 student s3 (54, "Alpha");
```

```
 student s4 (53);
```

```
 s1.display();
```

```
 s2.display();
```

```
 student s5;
```

```
 s5.display();
```

```
 return 0;
```

```
29
```



18/9/25



Experiment No. = 6

1. Single Inheritance

```
=> #include <iostream>
using namespace std;
```

```
class person {
protected:
 string name;
 int age;
```

```
};
```

```
class Derived : protected person {
```

```
private:
```

```
 int roll;
```

```
public:
```

```
 void accept() {
```

```
 cout << "enter name, roll no. & age
```

```
 cin >> name >> roll >> age;
```

```
 }
 void display() {
```

```
 cout << endl << "Name = " << name << endl
```

```
 "Roll no = " << roll << endl << "Age = "
```

```
 age << endl;
```

```
 }
```

```
int main() {
```

```
 Derived d1;
```

```
 d1.accept();
```

```
 d1.display();
```

### 3 > // Multi-level inheritance

→ #include <iostream>  
using namespace std;

class vehicle {

public:

String brand, model;

};

class car: public vehicle {

public:

String attribute;

};

class a car: protected car {

private:

String battery - capacity;

public:

void accept() {

cout << "Enter brand, car type, model  
Battery - capacity = " << endl <<

cin >> brand << attribute << battery -  
capacity << endl <<

void display() {

cout << endl << "car brand = " << brand << endl <<

"car type = " << attribute << endl << "car model = " <<

model << endl << "car battery = " <<

battery - capacity << endl << }

};

|          |     |
|----------|-----|
| Page No. |     |
| Date     | / / |

```

int main() {
 e car e1;
 e1.accept();
 e1.display();
 return 0;
}

```

1) // Hierarchical inheritance:

```

=> #include <iostream>
using namespace std;

class Employee {
public:
 string id, name;
};

class manager : protected Employee {
private:
 string dept;
public:
 void acc() {
 cout << "Enter Emp-id, Name, Dept = ";
 cin >> id >> name >> dept;
 }
 void disp() {
 cout << "Emp-id = " << id << "\n Name = " << name << "\n Emp dept = " << dept << "\n";
 }
};

```

Page No.   
 Date   
 Class developer: protected Employee {

private:

String programming - language = "Rust";

public:

void display() {  
cout << "programming language = "<<  
programming - language << endl;  
}

}

int main() {

manager m1;

developer d1;

m1.ac();

d1.display();

return 0;

}

5) // Hybrid inheritance

→ #include <iostream>

using namespace std;

class person {

protected;

String name;

int age;

}

class Sports &

public:

int Score;

void acc() {

cout << "Enter Sport Score = ";

cin >> Score;

};

};

class Academic &

public:

int marks;

void acc() {

cout << "Enter marks of student = ";

cin >> marks; }

};

class Student protected personal

public:

void accept() {

cout << "Enter age & name, St

cin >> age >> name; }

};

class result: protected Sports, Prote

Student, Protected Academic

public:

int total;



void disp() {  
Student: accept();  
academic: acc();  
Sports: acc();  
}

void calc() {  
total = marks + score;  
cout << endl << "Total marks of  
Student = " << total << endl;  
}  
};

int main() {  
result r;  
r1. disp();  
r1. calc();  
return 0;  
}

6.2 // virtual base class

```
#include <iostream>
using namespace std;
```

```
class college {
```

```
protected:
```

```
int id;
```

```
String name;
```

```
};
```

```
class test : virtual protected college {
```

```
public:
```

```
int per;
```

```
void accept() {
```

```
cout << endl << "Enter Student id n
```

```
name = ";
```

```
cin >> id >> name;
```

```
};
```

```
void accept() {
```

```
cout << endl << "Enter Student marks
```

```
= ";
```

```
cin >> per;
```

```
};
```

```
}; // sports : virtual Protected
```

```
class result : protected test, public
```

```
sports college {
```

```
public:
```

```
char grade;
```

```
void accept() {
```

```
cout << endl << "Enter grade = ";
```

|          |  |
|----------|--|
| Page No. |  |
| Date     |  |

```
3
3;
```

```
class result: protected test, Prote
```

```
Sports &
```

```
public:
```

```
void disp() &
```

```
test::acce();
```

```
test::accept();
```

```
Sports::accept();
```

```
3
```

```
3;
```

```
int main() &
```

```
result r;
```

```
r.disp();
```

```
return 0;
```

```
3
```

22/9/23

## Experiment - 7

1)

o #include <iostream>  
using namespace std;

float area (float length, float breadth)

{

return length \* breadth;

}

float area (float side)

{

return side \* side;

}

int main()

{

cout << "Area of rectangle is: " << area

cout << "Area of square is: " << area

return 0;

}

2)

```
#include <iostream>
using namespace std;
```

```
class Sum {
```

```
int number [10];
```

```
public:
```

```
void accept ()
```

```
{
```

```
cout << "Enter 10 integers:";
```

```
for (i = 0; i < 10; i++)
```

```
cin >> number [i];
```

```
}
```

```
int operator + (C)
```

```
{
```

```
int s = 0
```

```
for (int i = 0; i < 10; i++)
```

```
s = s + number [i];
```

```
return s;
```

```
}
```

```
}
```

```
int main ()
```

```
{
```

```
Sum s;
```

```
s.accept ();
```

```
cout << "Sum of 10 nos: << s,
```

```
return 0;
```

```
}
```



2)

```
→ #include <iostream>
using namespace std;
```

```
class number
```

```
{
```

```
 int num;
```

```
 public:
```

```
 void accept()
```

```
{
```

```
 cout << "Enter a number:";
```

```
 cin >> num;
```

```
}
```

```
 void display()
```

```
{
```

```
 cout << "Number:" << num << endl;
```

```
}
```

```
 void operator -()
```

```
{
```

```
 num = -num;
```

```
}
```

```
}
```

```
int main()
```

```
{
```

```
 number n;
```

```
 n.accept();
```

```
 -n;
```

```
 n.display();
```

```
 return 0;
```

```
}
```

4)

```
→ #include <iostream>
using namespace std;
```

```
class Number
```

```
{
 int num;
```

```
public:
```

```
 Number(int n)
```

```
{
 num = n; }
```

```
void operator++() // pre-increment
```

```
{
 num;
```

```
}
```

```
void operator++(int) {
 num++;
```

```
}
```

```
void operator++(int) {
 num++;
```

```
}
```

```
void display()
```

```
{
```

```
 cout << "value: " << num << endl;
```

```
}
```

```
}
```

```
int main()
```

```
{
 Number n(5)
```

```
 cout << "initial:"
```

```
 n.display;
```

```
 ++n
```

```

n. display c);
n++;
n. display c);
return 0;

```

2y

9  
10/11

## Experiment - 8

1)

→

```
#include <iostream>
#include <string>
using namespace std;
```

```
class myString
```

```
{
```

```
 string S;
```

```
 public:
```

```
 myString (string str)
```

```
{
```

```
 S = str;
```

```
}
```

```
 myString ()
```

```
{
```

```
 myString operator+(myString obj)
```

```
{
```

```
 return S = str + obj.S;
```

```
}
```

```
 void display()
```

```
{
```

```
 cout << S;
```

```
}
```

```
int main()
```

```
{
```

```
 myString a("xyz"), b("pqr"), c = a+b;
```

```
 cout << "addition of strings is =
```

```
 c.display();
```

```
}
```

2)

```
#include <iostream>
#include <string>
using namespace std;
```

```
class ILogin
```

```
{
```

```
protected:
```

```
 string name;
```

```
 string password;
```

```
public:
```

```
 virtual void accept()
```

```
{
```

```
 cout << "Enter name & password"
```

```
 cin >> name >> password;
```

```
}
```

```
 virtual void display()
```

```
{
```

```
 cout << "Name = " << name << endl;
```

```
 cout << "password = " << password;
```

```
}
```

```
}
```

```
class emailLogin : public ILogin
```

```
{
```

```
 string email;
```

```
 string password;
```

```
public:
```

```
 void accept() override
```

```
{
```

```
 cout << "Enter name email & password"
```



cin >> name >> email >> password;

}

virtual void display() override

{

cout << "Account Name = " << name;

cout << "Email = " << email;

cout << "password = " << password;

}

}

class member login : public Tlogin

{

String member id;

String password;

public

void accept() override

{

cout << "Enter name, member id & pass

cin >> name >> member id >> password;

}

virtual void display() override

{

cout << "Name = " << name;

cout << "member id = " << member id;

cout << "password = " << password;

}

}

int main()

{

Tlogin \* ptr;

emaillogin e;

member login m;

|      |  |
|------|--|
| NAME |  |
| DATE |  |

```
cout << "Email ID: "; member login = 2;
int disp();
cin >> choice;
```

```
if (choice == r)
```

```
ptr = &e;
```

```
else
```

```
ptr = &m;
```

```
ptr -> accept();
```

```
ptr -> display();
```

```
return 0;
```

```
}
}
```

12/11/2020

## Experiment - 9

1)

```
→ #include <iostream>, #include <fstream>
#include <string>
using namespace std;
```

```
int main ()
```

```
{
```

```
 fstream createFile ("first.txt", ios::out);
 if (!createFile)
```

```
{
```

```
 cout << "File not created" << endl;
```

```
}
```

```
else
```

```
{
```

```
 cout << "File created",
```

```
 createFile << "In This is sample text"
```

```
 createFile << endl;
```

```
}
```

```
 if (fstream input ("first.txt", ios::in);
```

```
 fstream output ("second.txt", ios::out);
```

```
 if (!input)
```

```
{
```

```
 cout << "File not created";
```

```
}
```

```
 if (!output)
```

```
{
```

```
 cout << "File not created";
```

```
}
```

32

```

-> #include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main()
{
 ofstream file ("data.txt");
 if (!file)
 {
 cout << "file not created:";
 }
 else
 {
 cout << "file created";
 file << "There is a impostor among us";
 file.close();
 }

 ifstream infile ("data.txt", ios::in);
 if (!infile)
 {
 cout << "file not created";
 }
 string word;
 int word count = 0;
 while (infile >> word)

```

while (infile >> word)

{

word count ++;

if (target == word)

{ target count ++; }

}

infile.close();

cout << "The total no. of occurrences  
target are << target count;

return 0;

}



## Experiment no. 10

```
#include <iostream>
#include <math>
using namespace std;

template <class T>
class P3
public:
 T m1, m2;

 void accept() {
 cout << "Enter the first number:" << endl;
 cin >> m1;
 cout << "Enter the second number:" << endl;
 cin >> m2;
 }

 void calc() {
 int choice;
 cout << "\n - Simple Calculator Menu - \n";
 cout << "1. addition \n";
 cout << "2. multiplication \n";
 cout << "3. division \n";
 cout << "4. Subtraction \n";
 cout << "5. Square root \n";
 cout << "6. percentage of m1 is what % of m2 \n";
 cout << "7. Power (Square of both numbers) \n";
 cout << "8. Trigonometric (sin values) \n";
 cout << "Enter your choice:" << endl;
```

|          |     |
|----------|-----|
| PAGE No. |     |
| DATE     | / / |

```

case 1:
cout << "sum = " << m1 + m2 << endl;
break;
case 2:
cout << "multiplication = " << m1 * m2 << endl;
break;
case 3:
cout << "division = " << m1 / m2 << endl;
break;
case 4:
cout << "subtraction = " << m1 - m2 << endl;
break;
case 5:
cout << "square root of " << m1 << " = " << sqrt(m1) << endl;
case 6:
cout << m1 << " to the power of " << m2 << " = " << pow(m1, m2) << endl;
break;
case 7:
cout << "square of " << m1 << " = " << pow(m1, 2) << endl;
cout << "square of " << m2 << " = " << pow(m2, 2) << endl;
break;
case 8:
cout << "sin of " << m1 << " = " << sin(m1) << endl;
cout << "sin of " << m2 << " = " << sin(m2) << endl;
break;

```

default:

```

cout << "Invalid choice: " << endl;
}
int main() {
 double m1, m2;

```

## Q2) Queue

```
#include <iostream>
#include <queue>
using namespace std;
template <class T>
class Queue {
private:
 queue <T> q;
public:
 void Enque (T elements) {
 q.push (Element);
 cout << "element << element << added \n";
 }
 void dequeue () {
 if (q.empty ()) {
 cout << "Queue is empty Cannot dequeue \n";
 }
 else {
 cout << q.front () << "deleted successfully \n";
 q.pop ();
 }
 }
 void display () {
 cout << "Queue elements: \n";
 Queue <T> temp = q;
 while (!temp.empty ()) {
 cout << temp.front () << " ";
 temp.pop ();
 }
 }
};
```

cout << endl;

};

};

int main() {

Queue q(10);

q.enqueue(10);

q.enqueue(20);

q.enqueue(30);

q.display();

q.dequeue();

q.display();

return 0;

};

Ques  
Ans

# Experiment no: 11

Q1)

```
#include <iostream>
#include <vector>
#include <ctype>
using namespace std;
int main() {
 vector<int> vec(5);
 int i;
 cout << "Enter vector 5 elements: ";
 for (i=0; i<5; i++) {
 cin >> vec[i];
 }
 cout << endl;
 cout << "vector elements are: <<endl;
 for (i=0; i<5; i++) {
 cout << vec[i] << endl;
 }
 cout << "modified elements are: ";
 for (i=0; i<5; i++) {
 for (i=0; i<5; i++) {
 cout << vec[i] << " ";
 }
 }
 cout << endl;
 int scalar;
 cout << "enter a scalar value to
 multiply: ";
 cin >> scalar;
 cout << "After multiplying by scalar"
```



# Experiment No: 12

|          |       |
|----------|-------|
| Roll No. |       |
| Date     | 11/11 |

```

1) #include <iostream>
#include <string>
#include <stack>
using namespace std;
template <class T>
class Stack {
Stack (T) obj - stack;
public:
void accept () {
int size;
cout << "Enter the size of the Stack"
<< endl;
cin >> size;
T data;
for (int i = 0; i < size; i++) {
cout << "Enter the data for the
Stack" << endl;
cin >> data;
my_stack.push(data);
}
}
void pop () {
T x;
cout << "Enter the value which you
want to delete in" << endl;
cin >> x;
if (my_stack.empty()) {
cout << "The Stack is empty" << endl;
}
}
}

```

if (my\_stack.empty()) {  
cout << "The stack is empty" << endl;  
}

if (!my\_stack.empty()) {  
my\_stack.pop();  
}

void display() {

if (my\_stack.empty()) {  
cout << "The stack is empty" << endl;  
}

while (!my\_stack.empty()) {  
cout << my\_stack.top();  
my\_stack.pop();  
}

}

}

}

}

}

}

}

}

}

}

}

}

}

}